

Reviewer Pack — Tropical Tensor-Product Factorization of MinimalFourierCoeff...

Entry 57df64f85242 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 21:31:08 UTC

Tropical Tensor-Product Factorization of MinimalFourierCoefficient

Entry ID: 57df64f85242 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-26 21:31:08 UTC

1. Conjecture

Field A (mathematical branch): TROPICAL_FOURIER_ANALYSIS framework (underlying branch: tropical geometry × harmonic / Fourier analysis on idempotent semirings) **Field B** (complexity object): Two-party communication complexity of distributed tropical Fourier coefficient evaluation (Yao model, max-plus oracle access)

Statement:

Let f be a TropicalPolynomial on variables x in $\{0,1\}^n$ and g be a TropicalPolynomial on disjoint variables y in $\{0,1\}^m$, both over the max-plus semiring. Define their tropical tensor product $(f \otimes g)(x,y) := f(x) + g(y)$ (which is a TropicalPolynomial obtained via TropicalConvolution on the disjoint-variable embedding). Then the FourierTransform satisfies the factorization identity, and the target invariant decomposes as: $\text{MinimalFourierCoefficient}(f \otimes g) = \text{MinimalFourierCoefficient}(f) + \text{MinimalFourierCoefficient}(g)$. Moreover, the DiscrepancyMeasure obeys the tensor inequality $\text{DiscrepancyCalculation}(f \otimes g) \leq \text{DiscrepancyCalculation}(f) + \text{DiscrepancyCalculation}(g)$, with equality whenever the MinimalFourierCoefficient is achieved on the all-zeros character of each factor.

Rationale (proposer's reasoning):

This stress-tests axiom A3 (DiscrepancyMeasure is bounded by the maximum absolute Fourier coefficient) together with axiom A1 (TropicalConvolution preserves semiring structure under composition). Because the tropical Fourier kernel is additively separable across disjoint variable blocks, the joint coefficient on a character (s,t) reduces to the tropical product of the marginal coefficients on s and t . Taking a min over (s,t) therefore distributes as a tropical sum of marginal minima, yielding an exact additive law for the target invariant. The Discrepancy inequality then follows from A3 since the joint maximum-coefficient envelope is sub-additive across independent blocks. Confirming the conjecture would supply a 'tensorization' rule analogous to the classical Plancherel product law, distinct from (and orthogonal to) the existing self-convolution doubling and convolution subadditivity sub-conjectures.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time:)

Framework membership: framework fw_28b4bfb95f, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 259a25a20a2f3d01

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 300 trials ($n=m=4$, coeffs Uniform $[-5,5]$), every trial must satisfy $|\text{MinFC}(f \otimes g) - (\text{MinFC}(f) + \text{MinFC}(g))| < 1e-9$ AND $\text{Disc}(f \otimes g) \leq \text{Disc}(f) + \text{Disc}(g) + 1e-9$; max observed deviation across all trials $< 1e-8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical Fourier transform max-plus semiring communication complexity
- idempotent harmonic analysis tensor product discrepancy two-party - min-plus polynomial
Fourier coefficient factorization Yao communication

Top relevant hits considered: - [http://arxiv.org/abs/1503.01392v2] Valuations of Semirings - [http://arxiv.org/abs/1010.5964v1] Quadratic discrete Fourier transform and mutually unbiased bases - [http://arxiv.org/abs/nlin/0505056v1] Max-Plus Algebra for Complex Variables and Its Application to Discrete Fourier Transformation - [http://arxiv.org/abs/1304.4922v1] An invitation to harmonic analysis associated with semigroups of operators - [http://arxiv.org/abs/1607.03608v3] On the tensor product of linear sites and Grothendieck categories - [http://arxiv.org/abs/1808.10521v2] Efficient quantum tensor product expanders and unitary t-designs via the zigzag product - [http://arxiv.org/abs/2512.17729v1] Fractionalized topological d+id superconductivity in the Yao-Lee-Kondo model - [http://arxiv.org/abs/1805.08186v5] On A Polytime Factorization Algorithm for Multilinear Polynomials Over F2

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json

def max_plus(a, b):
    return max(a, b)

def min_plus(a, b):
```

```

    return min(a, b)

def max_plus_fourier_transform(f, n):
    if n == 1:
        return f[0]
    half_n = n // 2
    even_part = max_plus_fourier_transform([f[2 * i] for i in range(half_n)], half_n)
    odd_part = max_plus_fourier_transform([f[2 * i + 1] for i in range(half_n)], half_n)
    result = [0] * n
    for k in range(n):
        if k < half_n:
            result[k] = max_plus(even_part[k], odd_part[(k - half_n) % half_n])
        else:
            result[k] = min_plus(even_part[k - half_n], odd_part[k - half_n])
    return result

def minimal_fourier_coefficient(f, n):
    fourier_transform = max_plus_fourier_transform(f, 2 ** n)
    return min(fourier_transform)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n, m = 4, 4
    f_coeffs = [random.uniform(-5, 5) for _ in range(1 << n)]
    g_coeffs = [random.uniform(-5, 5) for _ in range(1 << m)]

    def tropical_polynomial(f, x):
        return sum(f[i] * (-x[i]) for i in range(len(x)))

    f_conv_g = [0] * (1 << (n + m))
    for x in range(1 << n):
        for y in range(1 << m):
            f_val = tropical_polynomial(f_coeffs, list(bin(x)[2:].zfill(n)))
            g_val = tropical_polynomial(g_coeffs, list(bin(y)[2:].zfill(m)))
            f_conv_g[x * (1 << m) + y] = max_plus(f_val + g_val, -x - y)

    min_fc_f = minimal_fourier_coefficient(f_coeffs, n)
    min_fc_g = minimal_fourier_coefficient(g_coeffs, m)
    min_fc_fg = minimal_fourier_coefficient(f_conv_g, n + m)

    disc_f = max_plus_fourier_transform(f_coeffs, 2 ** n)[-1] - min_plus_fourier_transform(f_coeffs, 2 ** n)
    disc_g = max_plus_fourier_transform(g_coeffs, 2 ** m)[-1] - min_plus_fourier_transform(g_coeffs, 2 ** m)
    disc_fg = max_plus_fourier_transform(f_conv_g, 2 ** (n + m))[-1] - min_plus_fourier_transform(f_conv_g, 2 ** (n + m))

    conjecture_holds = abs(min_fc_fg - (min_fc_f + min_fc_g)) < 1e-9 and disc_fg <= disc_f + disc_g
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "DiscrepancyMeasure",
        "metric_value": disc_fg,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

```

```

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18e8ebc3.py", line 83, in <module>
 result = run_trial(seed)
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_18e8ebc3.py", line 55, in run\_trial  
 f\_val = tropical\_polynomial(f\_coeffs, list(bin(x)[2:].zfill(n)))  
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18e8ebc3.py", line 50, in tropical_poly
 return sum(f[i] * (-x[i]) for i in range(len(x)))
 ~~~~~

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_18e8ebc3.py", line 50, in <genexpr>  
 return sum(f[i] \* (-x[i]) for i in range(len(x)))  
 ~~~~~

TypeError: bad operand type for unary -: 'str'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` before any trial completed because binary digit characters were passed as strings to a unary minus operation, so no data on MinFC or Disc was produced. The pre-registered support condition cannot be evaluated. | next: Fix the tropical_polynomial harness by converting `bin()` digits to ints (e.g., `[int(b) for b in bin(x)[2:].zfill(n)]`) and rerun the 300 trials.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	5587
2	novelty	claude_max	opus	0	0	3226
3	novelty	claude_max	opus	0	0	9433
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12875
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11909
6	judge	claude_max	opus	0	0	4639

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 47668 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/57df64f85242.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/57df64f85242.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/57df64f85242.tar.gz` (if generated)